

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3084 - DATA SCIENCE

SECCIÓN – 40

Ing. Erick Marroquín



Laboratorio 5. Minería de Textos y Análisis de Sentimientos

Javier André Benítez García 23405

Bryan Alberto Martínez Orellana 23542

Adriana Sophia Palacios Contreras 23044

GUATEMALA, 30 de Agosto de 2026

1-2. Descripción y carga de los datos

El dataset tiene 5 columnas: id, keyword (palabra clave, con nulos), location (ubicación, con 33% de nulos y muy sucia), text (el tweet) y target (1 = desastre real, 0 = no). Se cargó con pandas desde data/raw/train.csv (src/01_ingesta.py).

3. Limpieza y preprocesamiento

El preprocesamiento se hizo midiendo primero el dataset (notebooks/analisis_pre.ipynb) y escribiendo después el pipeline, en lugar de aplicar la receta estándar a ciegas. Detalle completo de cada decisión en src/decisiones.md y codebook.md. Módulos usados: pandas, re, NLTK (stopwords, WordNetLemmatizer, SnowballStemmer).

Pasos aplicados, en orden (src/05_limpieza_texto.py y src/06_tokenizacion.py):

- Corrección de mojibake (\x89Ûª y similares), presente en 8.07% de los tweets y no resuelto por ftfy, así que se corrigió a mano.
- Extracción de señales del texto crudo ANTES de limpiar: presencia de URL, cantidad de hashtags, menciones y mayúsculas, porque la sola presencia de una URL ya es informativa (55% de tasa de desastre con URL contra 30% sin ella) aunque la cadena en sí no lo sea.
- Eliminación de URLs, menciones (@usuario), el prefijo "RT", el símbolo # (se conserva la palabra del hashtag) y minúsculas.
- Normalización de apóstrofes y expansión de contracciones (don't -> do not) antes de quitar la puntuación, para no dejar tokens basura como "s" o "t".
- Caso especial 911: se detectan las variantes 911, 9/11, 9-11 y WTC y se funden en un token único ANTES de quitar puntuación y números, porque si no la secuencia normal de limpieza lo destruye en dos pasos sin que se note. Aparece en 11 tweets, 8 de la clase 1.
- No se implementó eliminación de emojis: se midió que el dataset no tiene ni un solo emoji unicode. Los emoticones ASCII sí se conservan, aunque aparecen en menos del 1% de los tweets, así que no cambian ninguna métrica a nivel de corpus; se leen en el ejercicio 8 porque el léxico de VADER los incluye con sus signos.
- Eliminación de puntuación, números puros y stopwords de NLTK, y reducción de cada palabra a su raíz: dos versiones, lematización (WordNetLemmatizer) y stemming (SnowballStemmer), para poder comparar ambas en el modelo.

Se generan dos versiones del texto limpio y no una sola: texto_limpio (agresiva, para el modelo) y texto_sentimiento (conservadora, conserva mayúsculas/puntuación/negaciones, para el análisis de sentimiento de los ejercicios 8-10). La razón es que la limpieza agresiva quita "not" como stopwords, lo que invertiría el sentido de un tweet negado si se usara para sentimiento.

location se conserva en el dataset pero se descarta como predictora: 33% de nulos y ~3,341 valores únicos sobre 5,080 presentes, campo libre sin normalizar (USA, US, United States, bromas). Descartarla en la limpieza sería imponer una decisión de modelado antes de tiempo.

4. Frecuencia de n-gramas

Se generaron unigramas, bigramas y trigramas por clase (notebooks/4_frecuencias_tweets.ipynb), con frecuencia y probabilidad (frecuencia del n-grama entre el total de n-gramas de esa clase).

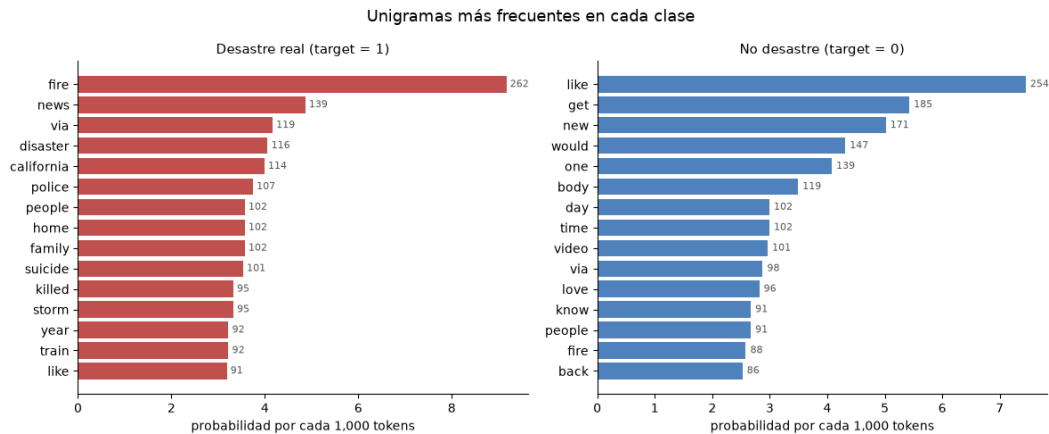


Figura 1. Unigramas más frecuentes por clase.

Como se observa en la Figura 1, la palabra más repetida en desastre real es fire y en no desastre es like. Para saber qué palabras separan mejor las clases no basta la frecuencia bruta: se calculó el log-odds (con suavizado de Laplace) de cada palabra con al menos 10 apariciones. Las de mayor log-odds hacia desastre son nombres de eventos concretos de 2015 (mh370, legionnaire, hiroshima), y palabras frecuentes en ambas clases casi por igual como emergency o fatality no aportan nada.

¿Vale la pena explorar bigramas o trigramas? En frecuencia de documento, solo el 11% de los bigramas y el 7.5% de los trigramas aparecen en 2 o más tweets distintos (contra 42% de los unigramas), y tomando los 200 más comunes de cada tipo, los unigramas cubren 84.8% del corpus contra 21.4% de los bigramas y 9.2% de los trigramas. Con un corpus de apenas 7,485 tweets cortos, los n-gramas largos son demasiado dispersos para generalizar: se usan como apoyo para leer contexto puntual, pero el modelo se entrena sobre unigramas (ver ejercicio 6).

5. Análisis exploratorio

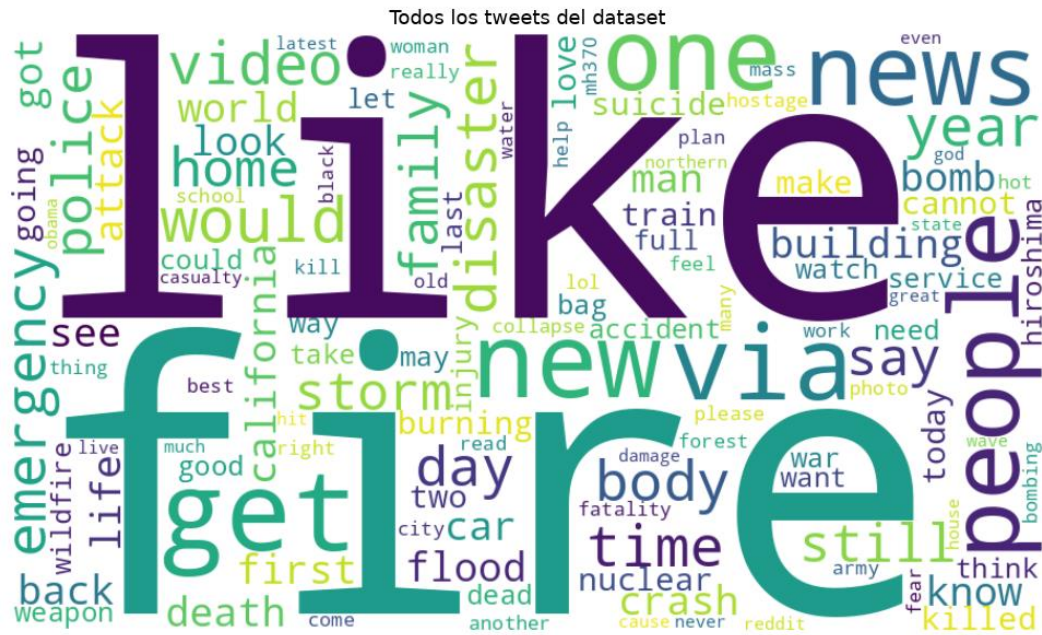


Figura 2. Nube de palabras por clase (desastre real / no desastre).

Las nubes de palabras de la Figura 2 separan los dos vocabularios a simple vista: del lado de desastre dominan fire, news, disaster, police, suicide y california; del otro like, get, new, would y body.

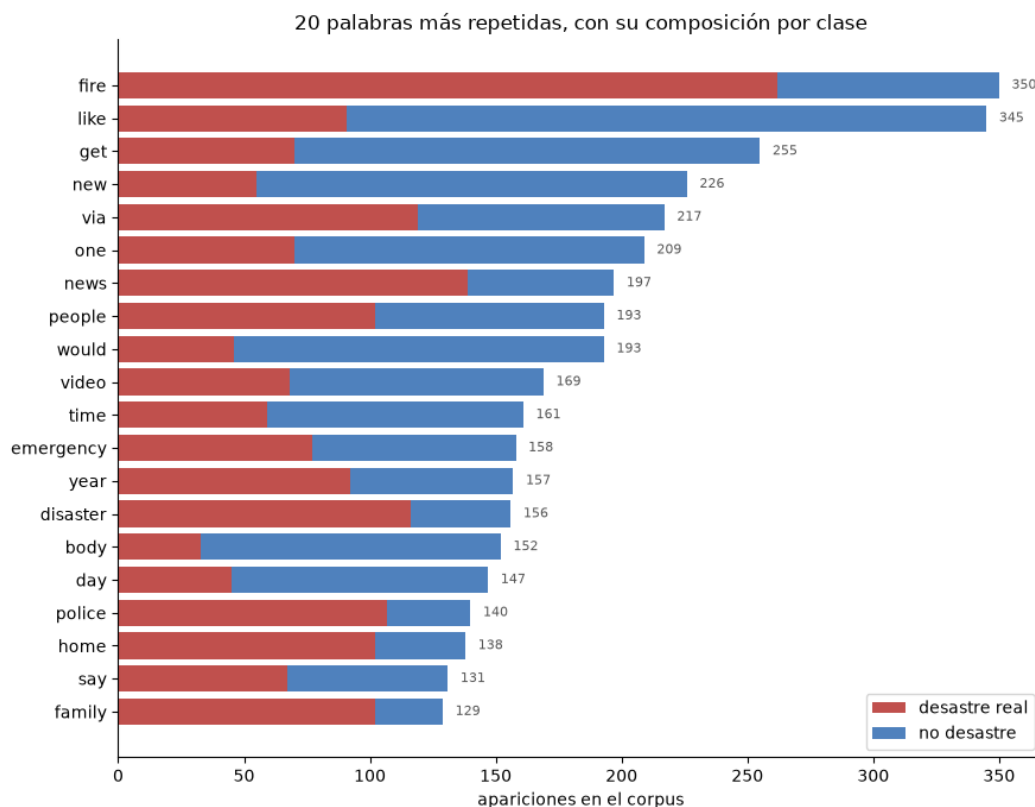


Figura 3. Las 20 palabras más repetidas del corpus, con su composición por clase.

Según el reparto por clase de la Figura 3, solo el 26.8% del vocabulario aparece en ambas categorías, pero esas palabras concentran el 76.6% de todo el texto. Palabras compartidas como *via*, *people* o *video* se reparten cerca de la tasa global (42.6%) y no discriminan nada; otras compartidas pero muy sesgadas, como *fire* (75% del lado desastre), sí discriminan aunque aparezcan en ambos lados. La conclusión práctica: estar presente en las dos categorías no descalifica a una palabra, lo que importa es el reparto.

6. Modelos de clasificación

Se probaron 3 algoritmos (regresión logística, Random Forest y una LSTM) sobre una rejilla de representaciones: lematización vs. stemming, unigramas vs. unigramas+bigramas, y bolsa de palabras vs. TF-IDF (la LSTM usa embeddings entrenados desde cero sobre secuencias de tokens, así que no entra en la rejilla completa). El contexto se abordó de dos formas: bigramas dentro de la bolsa de palabras, y orden explícito de la secuencia en la LSTM. Partición 70% entrenamiento / 30% prueba, estratificada por target semilla fija (src/08_particion.py) La Tabla 1 resume la mejor configuración de cada algoritmo.

Modelo	Texto	N-grama	Vectorizado r	Accurac y	Precisio n	Recall	F1
--------	-------	---------	------------------	--------------	---------------	--------	----

Random Forest	lematización	unigrama	bag of words	0.8059	0.7984	0.7283	0.7617
Regresión logística	stemming	unigrama	TF-IDF	0.8077	0.8269	0.6938	0.7545
LSTM	stemming	secuencia	embeddings	0.7743	0.7812	0.6531	0.7114
LSTM	lematización	secuencia	embeddings	0.7689	0.7731	0.6479	0.7049

Tabla 1. Mejor configuración de cada modelo sobre el conjunto de prueba.

Como muestra la Tabla 1, el mejor f1 lo dio Random Forest con lematización, unigramas y bolsa de palabras (300 árboles, min_samples_leaf=2, validado con 5-fold CV sobre entrenamiento). La regresión logística con stemming y TF-IDF queda a solo 0.0072 de f1, así que la diferencia entre ambos es frágil y no concluyente; se prefiere Random Forest porque detecta más desastres reales (recall 0.7283 contra 0.6938) a costa de un poco menos de precisión, y en este problema dejar pasar un desastre real es más costoso que una falsa alarma. La LSTM queda entre 0.04 y 0.05 de f1 por debajo del resto, consistente con lo visto en el ejercicio 4: con apenas 5,239 ejemplos de tweets cortos no hay datos suficientes para que aprenda embeddings y orden de secuencia desde cero.

7. Función de clasificación

clasificar_tweet(tweet) (notebooks/7_funcion_clasificacion.ipynb) recibe un tweet crudo, sin preprocesar, aplica exactamente la misma limpieza y tokenización de src/texto.py con la que se entrenó el modelo (Random Forest, lematización, unigramas, bolsa de palabras) y devuelve DESASTRE o NO DESASTRE, con la probabilidad si se pide detalle. Reutiliza la limpieza en lugar de reescribirla para que la función y el modelo evaluado nunca diverjan.

8. Clasificación de sentimiento (positivo / negativo / neutro)

Se usó VADER (módulo vaderSentiment) en vez de TextBlob porque su léxico y sus reglas están hechas para texto de redes sociales: pondera mayúsculas, puntuación repetida, negaciones e intensificadores. La función clasificar_sentimiento(tweet) (notebooks/8_9_10_sentimiento.ipynb, también corre como etapa 9 del pipeline en src/09_sentimiento.py) cuenta cuántas palabras del tweet están en el léxico de VADER con signo positivo o negativo, calcula el puntaje compuesto (compound, en [-1, 1]) y clasifica: compound >= 0.05 positivo, <= -0.05 negativo, si no neutro.

Se corrió sobre texto_sentimiento (limpieza conservadora, no la agresiva del modelo de clasificación), porque VADER necesita mayúsculas, puntuación y negaciones intactas para funcionar. Sobre dejar los emoticones: ya se había medido en el ejercicio 3 que aparecen en menos del 1% de los tweets y que no hay emojis unicode en el dataset.

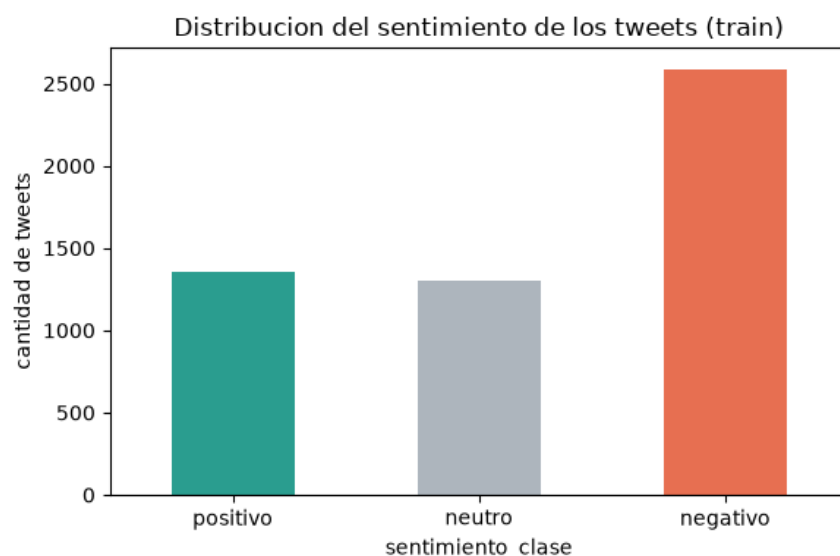


Figura 4. Distribución del sentimiento de los tweets.

Como muestra la Figura 4, el 49.4% de los tweets quedó como negativo, 25.9% positivo y 24.7% neutro, lo cual es esperable dado el tema del corpus: incluso tweets que no hablan de un desastre real usan a menudo palabras cargadas negativamente (fire, crash, wreck en sentido coloquial).

9. Tweets más negativos / más positivos

9.1 Los 10 tweets más negativos

De los 10 tweets con menor compound, 7 pertenecen a la categoría de desastre real (target = 1) y solo 3 no. Ejemplos: "Suicide bomber kills 15 in Saudi security site mosque..." (compound - 0.9623) y "17 killed in S'Arabia mosque suicide bombing" (-0.9552).

9.2 Los 10 tweets más positivos

De los 10 con mayor compound, 9 son de la categoría "no desastre" y 1 sí lo es. Ejemplos: "Check out 'Want Twister Tickets AND A VIP EXPE..." (compound 0.973) y "Super sweet and beautiful :)" (0.930).

9.3 ¿Son los tweets de desastre real más negativos?

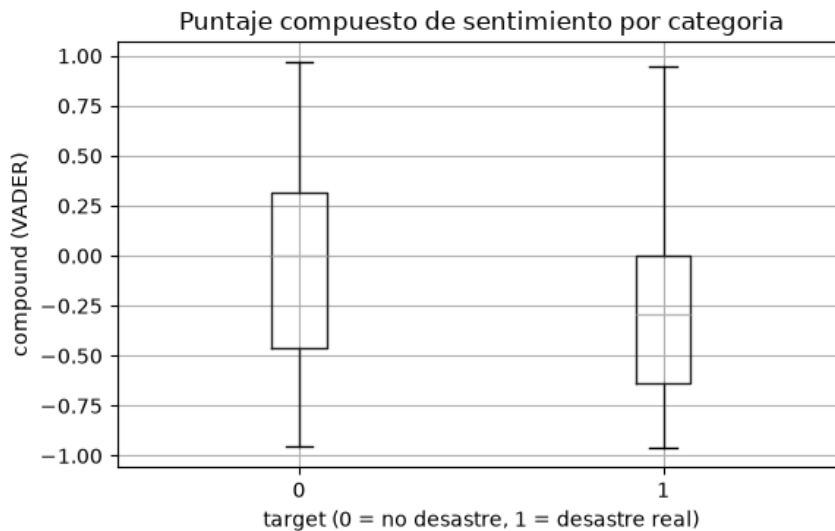


Figura 5. Puntaje compuesto de sentimiento por categoría (target).

Categoría	Compound promedio	Negatividad promedio
No desastre (target=0)	-0.0527	0.1396
Desastre real (target=1)	-0.2682	0.1853

Tabla 2. Sentimiento promedio por categoría.

Sí: como resume la Tabla 2, en promedio los tweets de desastre real son más negativos (compound -0.27 contra -0.05, negatividad 0.18 contra 0.14). La diferencia es moderada, no extrema, y hay bastante traslape entre las dos distribuciones (ver Figura 5), es decir que el sentimiento por sí solo no separa las clases con precisión, y por eso el ejercicio 10 lo prueba como variable adicional del modelo y no como regla de clasificación aparte.

10. Variable de negatividad en el modelo

Se definió negatividad como el componente "neg" de VADER, en el rango [0, 1], calculado sobre texto_sentimiento (src/09_sentimiento.py, columna incluida en 09_sentimiento_train.csv y 09_sentimiento_test.csv). Se reentrenó la misma configuración ganadora del ejercicio 6 (Random Forest, lematización, unigramas, bolsa de palabras) agregando negatividad como columna extra a la matriz de bolsa de palabras (scipy.sparse.hstack), sin tocar el resto del pipeline.

Modelo	Accuracy	Precision	Recall	F1
Sin negatividad	0.8059	0.7984	0.7283	0.7617
Con negatividad	0.7992	0.8100	0.6907	0.7456

Tabla 3. Efecto de agregar la variable de negatividad al modelo.

Como se observa en la Tabla 3, la variable no mejoró el modelo: el f1 bajó de 0.7617 a 0.7456 (-0.0161) y la accuracy de 0.8059 a 0.7992. Sube apenas la precision pero cae el recall. Es

consistente con lo visto en el ejercicio 9: el sentimiento separa las clases solo de forma parcial y con bastante traslape, así que negatividad no le aporta información nueva a un bosque de 300 árboles que ya ve miles de columnas de vocabulario construidas sobre el mismo texto: un único número extra compite en poca proporción y su señal se diluye en vez de reforzar la decisión.

11. Conclusiones generales

- El preprocesamiento se decidió midiendo el dataset primero (src/decisiones.md), lo cual llevó a tres decisiones que contradicen la receta estándar del enunciado: no quitar emojis (no hay ninguno), conservar los emoticones ASCII aunque sean menos del 1% porque VADER los interpreta y conservar el 911.
- Los unigramas cubren el corpus mucho mejor que bigramas o trigramas (84.8% contra 21.4% y 9.2% con los 200 más comunes de cada uno), así que el modelo final se entrena sobre unigramas y los n-gramas largos quedan como apoyo de lectura de contexto.
- El mejor modelo de clasificación es Random Forest con lematización, unigramas y bolsa de palabras ($f1 = 0.7617$), aunque su ventaja sobre regresión logística con stemming y TF-IDF ($f1 = 0.7545$) es menor que la variación que produce cambiar la partición de datos.
- El análisis de sentimiento con VADER confirma que los tweets de desastre real son, en promedio, más negativos (compound -0.27 contra -0.05), pero la señal no es lo bastante limpia como para mejorar el modelo: agregar negatividad como variable bajó el $f1$ de 0.7617 a 0.7456.

12. Referencias

Bird, S., Klein, E., & Loper, E. (2009). Natural language processing with Python: Analyzing text with the Natural Language Toolkit. O'Reilly Media.

Howard, A., devrishi, Culliton, P., & Guo, Y. (2019). Natural language processing with disaster tweets [Conjunto de datos]. Kaggle. <https://kaggle.com/competitions/nlp-getting-started>

Hutto, C. J., & Gilbert, E. (2014). VADER: A parsimonious rule-based model for sentiment analysis of social media text. Proceedings of the Eighth International AAAI Conference on Weblogs and Social Media (ICWSM-14), 216–225.

Jurafsky, D., & Martin, J. H. (2024). Speech and language processing (3.^a ed., borrador). <https://web.stanford.edu/~jurafsky/slp3/>

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32, 8024–8035.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.